

Admin-portal parity programme

Sixteen build TABs, twenty gaps closed, and three reasons this is not a certified system. Amended after publication.

NOT CERTIFIED. Every gap is closed in this repository. Certification is a claim about a working system, and nothing is deployed for this one to speak to.

PRODUCT	MEASURED	ADMIN LINE	SUITE
eBPCO Mobile — applicant app	28 August 2026	Upupapp/eBPCO-Web @ e0925d9	1544 tests, analyzer clean

eBPCO Mobile — Admin-Portal Parity Programme

Closing certification

Verdict: NOT CERTIFIED. *(Amended 28 August 2026 — see §3; B-1 and B-2 were re-measured against the backend repository after publication. The verdict is unchanged.)*

Every one of the sixteen build TABs is complete, all twenty standing gaps are closed in this repository, and the suite is green at 1544 tests. The programme is not certified because certification is a claim about a working system, and three things stood between this repository and one. Two of them — no deployed backend, and routes the app calls that do not exist — are the backend lane's and remain open. The third, an offline queue built and wired to nothing, **was** this lane's, and was closed on 29 August; what is left of it is two triggers that are decisions rather than work. See §3.

Measured 28 August 2026 against the admin portal at [Upupapp/eBPCO-Web](#) [e0925d9](#) (26 August 2026), and this repository at the commit that carries this document.

1. What was measured

The comparison that produced the master command was re-run against the admin line as it stands today, mechanically rather than by reading.

`scripts/extract_admin_vocabulary.mjs` reads the admin's `src/app/core/domain/` and prints its nine closed vocabularies — 56 values — as JSON. `scripts/sync_admin_vocabulary.sh` stamps the admin's current commit and date onto the result and vendors it at `test/contract/admin-vocabulary.json`.

`test/contract/admin_vocabulary_extracted_test.dart` asserts this app against it, value for value and in order.

This matters because the fixtures that existed before today were **transcribed by hand**. A hand transcription is a claim about a file, and this programme was wrong about what a file said three times: twice by regex over `requirements-catalog.ts`, and once by quoting an admin comment that described the vendor's mobile copy rather than this one. The transcribed fixture is kept beside the extracted one deliberately. If the two ever disagree, one of them is wrong about the admin, and finding out which is the point.

Drift since 25 August

The admin moved from `e3cd7c3` to `e0925d9` — one commit, "Correct the product name to Building Permit and Certificate of Occupancy", 26 August. `git diff e3cd7c3..e0925d9 -- src/app/core/domain/` is empty: **no vocabulary drifted**. All nine vocabularies extract identically to the transcription made on 27 August.

One divergence found, in the admin

The admin declares the assessment-status vocabulary twice, and the two declarations disagree:

Declaration	Order
<code>type AssessmentStatus</code>	... Partially Paid, Paid , Overdue , Superseded ...
<code>const ASSESSMENT_STATUS_ORDER</code>	... Partially Paid, Overdue , Paid , Superseded ...

Nothing in the admin references `ASSESSMENT_STATUS_ORDER`. The union is the definition, and this app follows it. Both are now extracted into the fixture so the contradiction is visible rather than hidden by whichever one a script happened to read, and a test asserts that they still disagree — so the day someone reconciles them, it fails and says which order won, rather than this app quietly matching the loser.

Owner: admin lane. Cosmetic today; it becomes a real defect the moment anything sorts by it.

2. The gap register

Twenty gaps stood when the programme began; one more was withdrawn on inspection. All twenty are closed in this repository.

ID	Gap	Closed by	Artefact
G-01	Per-document review status absent	TAB 02	<code>DocumentModel.status</code>
G-02	Document rejection remarks never reach the applicant	TAB 02	<code>DocumentModel.remarks</code>
G-03	Payment rejection reason has nowhere to display	TAB 06	<code>PaymentTransactionRecord.rejectionReason</code>
G-04	"Partially Paid" unrepresentable	TAB 06	<code>PaymentAssessmentStatus.partiallyPaid</code>
G-05	Official Receipt not modelled	TAB 07	<code>orNumber</code> / <code>orDate</code> / <code>orIssuedBy</code>
G-06	Zoning / Locational Clearance cannot be filed	TAB 03	<code>zoning_clearance/</code> , 5 steps, 16 documents
G-07	FSEC cannot be filed	TAB 04	<code>fsec_clearance/</code> , 4 steps, 9 documents
G-08	FSIC cannot be filed	TAB 05	<code>fsic_clearance/</code> , 4 steps, 10 documents
G-09	No requirements catalog	TAB 01	<code>requirements_catalog.dart</code> , 19 types, 171 documents
G-10	Permit validity / expiry absent	TAB 09	<code>ApplicationModel.expiryDate</code>
G-11	Release method & claimant not modelled	—	Withdrawn: the admin comment described the vendor's mobile copy
G-12	Collecting agency not distinguished	TAB 07	<code>CollectingAgency</code>
G-13	Assessment supersession invisible	TAB 08	<code>supersededOrders</code> , <code>wasReassessed</code>
G-14	Fee line items not itemised from a versioned assessment	TAB 08	<code>OrderOfPayment.version</code> + <code>AssessmentFees</code>
G-15	Official blank forms & checklists unavailable	TAB 10	18 bundled PDFs, ~6.6 MB
G-16	Contact verification absent	TAB 11	<code>ContactVerification</code> , per channel
G-17	Citizen's Charter covers 5 types of 19	TAB 12	all 19, asserted
G-18	Document issuing office / issue & expiry dates not captured	TAB 02	<code>issuingOffice</code> , <code>issueDate</code> , <code>expiryDate</code>
G-19	Payment adjustments not modelled	TAB 06	<code>PaymentAdjustmentRecord</code>

ID	Gap	Closed by	Artefact
G-20	No renewal or amendment flow	TAB 14	ApplicationLineage

3. Why NOT CERTIFIED

AMENDED 28 August 2026, after publication. B-1 and B-2 were re-measured against the backend repository `~/ebpco-api` at `6138996`, which this section had not accounted for. The verdict does not change; two of its reasons do, and the amended text stands below with the original preserved beneath each.

B-1 — A backend exists and is reachable by nobody. Owner: backend lane.

`~/ebpco-api` has **86 routes**. What has not happened is a deployment: `AppConfig.apiUrl` is a `String.fromEnvironment` with **no default**, so `RepositoryFactory` still returns the mock implementations, and `HttpApplicationsRepository` is still reached by nothing in a running build. Nothing above can be verified against a live system until an instance exists somewhere.

This is still M-27, and it is still a decision as much as a task — but the decision left is **hosting and who operates it**, not the technology, which the backend repository has settled.

Originally published as "There is no backend. The app has never spoken to a server." The second sentence remains true; the first was wrong.

B-2 — Two of the three routes the app calls still do not exist. Owner: backend lane.

	Route	State
Document resubmission	<code>POST /applications/:id/documents/:documentId/resubmit</code>	M-43 open
Renewal / amendment reference	<code>renews_permit_id</code> on <code>POST /applications</code>	M-44 closed <code>e60cf3f</code>
Contact verification	send and confirm, per channel	M-45 server half only

M-44 is genuinely closed — a `renews_permit_id` foreign key plus `renewsPermitNumber` on both filing paths, refusing a renewal that names nothing, a permit that is not the applicant's, or a New application claiming to renew, with the database constraint enforcing it independently.

M-45 has a server half and no delivery. `GET /me/contacts` and `POST /me/contacts/:channel/request|confirm` exist and use the admin's exact wire strings. There is no email or SMS provider, so `request` answers `202` with `delivery: "not-sent"` and **no human can reach the success path** — the server's tests plant a digest to get past it. That is the same honest refusal the app makes, and it is not a closure.

M-43 is open, and was briefly and wrongly recorded as closed. The app makes

two resubmit calls and only one has a server:

- `POST /applications/:id/instructions/:letterId/resubmit` — **exists**

(`applicant-write.controller.ts:249`), the Letter of Instruction loop.

- `POST /applications/:id/documents/:documentId/resubmit` — **absent**, and

undeclared in the contract too.

They are different operations: a document can be rejected with remarks when no Letter of Instruction exists at all, which is why TAB 02 built the per-document loop. A search for "resubmit" matches the first and reads as closure — the failure mode this programme has hit repeatedly, where a lookup *resolves plausibly* instead of failing.

`~/ebpco-contract/tools/verify.sh` has said so throughout and still does:

"the server has no route for: mobile calls `POST /applications/{param}/documents/{param}/resubmit`". **Trust the gate over the grep.**

B-3 — Wired 29 August. Largely closed; two triggers remain a decision.

Amended. As published this read: " *OfflineQueue* and *SyncEngine* are constructed **nowhere** in *lib/* " — measured, and true at the time. The consequence it drew was that TAB 11's criterion, "*verification requests survive a failed network call*", was satisfied by the provider and proven by a test that handed it a queue, while the running app had no queue to give it: real code with no live wiring.

Both are now constructed. *SyncProvider* owns a *SecureQueueStore* -backed *OfflineQueue* , the contact-verification provider is handed that same queue rather than null, and a flush runs on app resume. The assertion in *certification_claims_test.dart* that guarded this claim **fired on the day it changed, naming both files** — which is what it was for — and has been inverted: it now fails if the wiring is ever removed.

What is still open, and why it is not this lane's to close:

Trigger	Blocked on
On regaining connectivity	<i>connectivity_plus</i> — a new dependency in a public LGU app, the owner's call. A <i>ConnectivityMonitor</i> seam exists so it is one class, not a refactor
While backgrounded	A background-execution entitlement, which changes what Apple reviews

And a flush has nowhere to send to until B-1 is answered, so with no API configured every item stays queued — the honest outcome, and asserted as such rather than allowed to look like success. Four of the five operation kinds throw rather than replaying, deliberately: a *_send* that silently succeeded would drop an applicant's work while reporting it delivered.

M-39 accordingly narrows from "wire it" to "decide the two triggers".

4. What is open, and who owns it

Backend lane

Item	What it needs	State
M-27	Deploy it somewhere and decide who operates it. The technology is settled; nothing is deployed	Open
M-43	Provide the per-document resubmission route, and declare it in the contract	Open
M-44	Carry the renewal reference on <code>POST /applications</code>	Closed <code>e60cf3f</code>
M-45	An email and SMS provider. The routes exist; delivery does not, so no applicant can complete a verification	Half open

Contract `0.2.0` was published from this programme (TAB 13): four notification types for states the applicant could reach and nobody could announce. Its client-alignment check reports three problems, all predating it.

Admin lane

Item	What it needs
Assessment status order	Reconcile the union with <code>ASSESSMENT_STATUS_ORDER</code> , or delete the unused constant
Requirements catalog enrichment	Twelve trade wizards collect more than the catalog lists. The wizards are not wrong — the catalog is thin. Recorded in <code>docs/REQUIREMENTS-DRIFT-2026-08-27.md</code>
Renewal requirements	The LGU publishes no shorter list for renewals. This app deliberately does not invent one; the office must state it

Decisions, deliberately not taken

Item	Why it is not this lane's
Whether contact verification ever gates filing	A product decision for the LGU. TAB 11 surfaces the state and gates nothing
M-40 — formatting the whole tree	Needs both platform lanes to land it in the same moment
M-04 — pushing <code>main</code> to <code>Upupapp/eBPCOMobile</code>	The owner's to run

5. What this programme is confident about

Against the admin's own source, extracted today:

- **All 19 permit types are filable**, with the office's exact names — including

the EN DASH that has already broken one route.

- **Nine closed vocabularies match**, in order, and parsing is total and closed:

every value the admin can send round-trips, and an unrecognised one throws rather than defaulting to something plausible.

- **1544 tests**, `flutter analyze` clean, zero layout overflows at 2.0× text

scale across every routable screen.

And one habit, worth more than any of them: **a green result is a claim that must be falsified before it is believed**. This programme produced eight confident claims that did not survive measurement — five wizards filing the wrong permit type, a route sweep that rendered the splash screen fifty times, a scanner blind to 127 anonymous closures, three wrong counts of the same upload slots, a charter route dead on the app's three most common filings. Each was found by trying to break the thing that had just passed.

The gates added along the way exist so the next one fails a test instead of reaching an applicant.